5.3 Estrutura de Decisão Simples

Uma estrutura de decisão (ou desvio condicional) pode ser traduzida diretamente num comando ou instrução: **SE**.

Uma **estrutura de decisão simples (ou desvio condicional simples)** é usada quando se deseja executar **um comando** ou um **bloco de comandos {...}, dependendo do resultado** da avaliação **de** uma **expressão relacional** ou **lógica**, que pode ser: **Verdadeiro** ou **Falso**.

✍ **A sintaxe dessa estrutura, em descrição narrativa, pode ser da seguinte forma:**

```
P9:  <instrução x>;
P10: Verifique SE  <condição>   então
        P10.1: <instrução1>;            // para a condição Verdadeira
P11: <instrução 2>;                     // instrução incondicional
```

A **instrução1** será executada **somente se a <condição> for Verdadeira**. Lembrando que <condição> pode ser um expressão relacional ou lógica.

A **instrução2** será executada **independentemente da <condição> ser Verdadeira** ou Falsa.

Na sintaxe acima podemos entender que **somente uma instrução** será executada quando a **<condição> for Verdadeira.**

☞ Quando for necessário **executar mais de uma instrução quando a <condição> for verdadeira**, usa-se a seguinte **sintaxe**:

```
P9:  <instrução x>;
P10: Verifique SE  <condição>   então
      Início de bloco
        P10.1: <instrução1>;
        P10.2: <instrução2>;          Bloco Verdade
        P10.2: <instrução3>;
      Fim de bloco;
<Instrução4>;    // instrução incondicional
```

As instruções a serem executadas quando a <condição> for Verdadeira devem estar escritas **dentro de um Bloco**, denominado **Bloco Verdade.**

☞ Na que na linguagem C++ um bloco de comandos é **delimitado pelo uso das chaves {...}.**

Sintaxe das Estruturas de Decisão Simples, em linguagem C++:

```
if <condição>
        <instrução1>;          // para a condição Verdadeira
<instrução2>;                  // instrução incondicional
```

```
if  <condição>
    {                          // Início do bloco Verdade
        <Instrução1>;
        <Instrução2>;
        <Instrução3>;
    }                          // Fim do bloco Verdade
<Instrução4>;    // instrução incondicional
```

Exemplo 1:

Para exemplificar o uso da estrutura de decisão simples, será considerando o algoritmo da aula anterior: **Calcular quanto tempo uma pessoa está casada**.

Após cálculo do tempo de casado será construída uma estrutura de decisão simples para verificar o tempo de casado e mostrar uma das mensagens a seguir, quando:

1. **Tempo de Casado = 25** - mensagem: **Parabéns! Você está comemorando BODAS DE PRATA!**
2. **Tempo de Casado = 50** - mensagem: **Parabéns! Você está comemorando BODAS DE OURO!**
3. **Tempo de Casado = 75** - mensagem: **Parabéns! Você está comemorando BODAS DE BRILHANTE!**
4. **Tempo de Casado = 100** - mensagem: **Parabéns! Você está comemorando BODAS DE OSSO!**

A solução algorítmica, na forma de uma descrição narrativa pode ser dada pela **sequência de ações abaixo**:

Algoritmo: **Calculo do Tempo de Casado de uma pessoa.**

Início

Passos:
P1: Solicite o Ano do Casamento;
P2: Considere o Ano Corrente ou Atual;
P3: Calcule Tempo de Casamento = Ano Corrente – Ano do Casamento;
P4: Escreva Tempo de Casamento;
P5: Verifique SE Tempo de Casamento = 25 então
P5.1 Mostre a mensagem 1 acima
senão
P5.2 SE Tempo de Casamento = 50 então
P5.2.1 Mostre a mensagem 2 acima
senão
P5.3 SE Tempo de Casamento = 75 então
P5.3.1 Mostre a mensagem 3 acima
senão
P5.4 SE Tempo de Casamento = 75 então
P5.4.1 Mostre a mensagem 4 acima

Fim.

A solução algorítmica para o exemplo anterior, na forma de um programa em linguagem C++ é mostrada a seguir.

```
#include<stdio.h>
#include<stdlib.h>
#include<locale.h>

// Exemplo usando Estrutura de Decisão Simples

main()
{

        setlocale(LC_ALL,"Portuguese");

// Definição de variáveis

    int AnoCasamento, AnoAtual, TempoCasado;

    system("cls");
    printf("\n<<  Cálculo do Tempo de Casado de uma Pessoa  >>");

// entrada de dados

    printf("\n\n Informe Ano do seu Casamento: ");
    scanf("%d",&AnoCasamento);

    printf("\n Informe Ano Atual.....: ");
    scanf("%d",&AnoAtual);

// processamento do cálculo

        TempoCasado = AnoAtual - AnoCasamento;

// saída de dados

        system("cls");
        printf("\n<<  Cálculo do Tempo de Casado de uma Pessoa  >>");

        printf("\n\n Ano do seu Casamento: %d",AnoCasamento);
        printf("\n Ano Atual.....: %d",AnoAtual);
        printf("\n\n Você está casado há: %d Anos.\n",TempoCasado);

        if (TempoCasado == 25)
           printf("\n\n Parabéns! Você está comemorando BODAS DE PRATA!\n");
        else
             if (TempoCasado == 50)
                printf("\n\n Parabéns! Você está comemorando BODAS DE OURO!\n");
             else
                  if (TempoCasado == 75)
                       printf("\n\n Parabéns! Você está comemorando BODAS DE BRILHANTE!\n");
                  else
                       if (TempoCasado == 100)
                            printf("\n\n Parabéns! Você está comemorando BODAS DE OSSO!\n");

        return 0;
}
```

1. Exercícios de fixação de conceitos sobre: Estrutura de Decisão Simples

Elaborar algoritmos para solucionar os problemas abaixo, usando a linguagem C++:

1. Solicitar e receber dois valores inteiros, e independentemente da ordem em que foram entrados, mostrá-los em ordem crescente.

2. Solicitar e receber o ano de nascimento de uma pessoa, calcular e mostrar sua idade e, também, verificar e mostrar se ela já tem idade para votar (16 anos ou mais).

3. Solicitar e receber um valor expresso em grau Fahrenheit. Converter esse valor para graus Celsius (ºC), mostrando o resultado dessa conversão. Em seguida, verificar se a temperatura convertida para Celsius for igual ou maior que 100 escrever as mensagens a seguir, em linhas distintas na tela do computador:

   "Foi atingida a temperatura de ebulição", e

   "Baixar a Temperatura".

   A fórmula para conversão é: ºC = 5 * (ºF - 32) / 9.

4. Calcular e mostrar a velocidade média de um veículo, sendo informado e recebido os seguintes dados de entrada: a distância entre duas cidades quaisquer, e o tempo de duração da viagem (em horas).

   Verificar se a velocidade média for maior que 120 escrever as mensagens a seguir, em linhas distintas na tela do computador: "Você rodou em velocidade acima do limite.", "Sujeito a multas.".

5. Calcular e mostrar a nota final de um aluno, composta por três notas: uma prova e dois trabalhos. O critério para o cálculo da nota final é o seguinte:

   Nota final = (Nota Prova x 0,80) + ((Nota Trabalho1 + Nota Trabalho2)/2 x 0,20).

   Verificar se a nota final apurada for igual ou maior a 7 escrever a mensagem:
   "Você está Aprovado!!!".

Observação: Nesses exercícios, os dados de entrada serão informados pelo teclado, e os de saída serão exibidos em monitor de vídeo.

4.4 Estrutura de Decisão Composta

Uma **estrutura de decisão composta** (ou desvio condicional composto) é usada quando se deseja **executar um comando, ou um bloco de comandos,** quando o **resultado da avaliação de uma expressão relacional ou lógica** for **Verdadeiro**, ou **outro comando ou bloco de comandos** quando **o resultado dessa avaliação** for **Falso.**

A sintaxe dessa estrutura, em descrição narrativa, pode ser da seguinte forma:

```
P9:  <instrução x>;
P10: Verifique SE  <condição>   então
        P10.1: <instrução1>          // para a condição Verdadeira
     senão
        P10.2: <instrução 2>;        // para a condição Falsa
```

A i**nstrução1** será executada somente **se** a **<condição>** for **Verdadeira**, enquanto que a **instrução2** será executada **se** a **<condição>** for **Falsa**.

Na sintaxe acima podemos entender que **somente uma instrução será executada quando a condição** (resultado de uma expressão relacional ou lógica) for **Verdadeira**, e **somente uma instrução será executada quando a condição** for **falsa.**

Observação: Essa estrutura inicia-se com a **verificação** (SE), e termina no último **ponto-e-vírgula (;)**, **que encerra,** no caso o da **instrução 2.**

Observar que **antes da palavra "senão" não tem ponto-e-vírgula (;)**, pois caso este seja colocado irá encerrar a verificação, entendo-se que a estrutura não é composta e, portanto, a instrução que vem após o "senão" não estará ligada a estrutura, e isso é errado.

A linguagem **C++ aceita essa sintaxe**, todavia, em **outras linguagens isso provocará erro de sintaxe**.

Quando for necessário **executar mais de uma instrução** (bloco de comandos) **quando a condição** for **verdadeira** ou **falsa**, usa-se a seguinte **sintaxe**:

```
P9:  <instrução x>;
P10: Verifique SE  <condição>
        Inicio                       // Inicio do Bloco Verdade
            P10.1: <Instrução1>;
            P10.2: <Instrução2>;
            P10.3: <Instrução3>;
        Fim                          // Fim do Bloco Verdade
     senão
        Inicio                       // Inicio do Bloco Falso
            P10.4: <Instrução4>;
            P10.5: <Instrução5>;
            P10.6: <Instrução6>;
        Fim;                         // Fim do Bloco Falso
```

Observação - Observar que a sintaxe da estrutura do Bloco Verdade é encerrado pela palavra **FIM sem um ;** (ponto e vírgula), pois **o comando é composto.**

Agora, toda a estrutura de decisão **encerra-se no último ponto e vírgula após** a palavra **FIM;** do **Bloco Falso**.

As instruções a serem executadas quando a condição for Verdadeira ou Falsa devem estar escritas dentro de um Bloco, denominado **Bloco Verdade** ou **Bloco Falso**, **delimitado pelo uso das instruções INICIO** e **FIM;**

Em linguagem C++ obloco de comando é **delimitado pelo uso das chaves {...}.**.

Sintaxe das Estruturas de Decisão Composta, em linguagem C++:

```
if <condição>
        <instrução1>;        // para a condição Verdadeira
else
        <instrução2>;        // para a condição Falsa
```

✓ Conforme já foi dito, a linguagem C++ aceita **ponto-e-vírgula (;) antes** da palavra **"else".**

```
if  <condição>
    {                        // Início do bloco Verdade
       <Instrução1>;
       <Instrução2>;
       <Instrução3>;
    }                        // Fim do bloco Verdade
else
    {                        // Início do bloco Falso
       <Instrução4>;
       <Instrução5>;
       <Instrução6>;
    }                        // Fim do bloco Falso
```

☞ **Exemplos em linguagem C++:**

2. Receber dois valores inteiros: valor1 e valor 2. Efetuar a soma desses valores resultando em um terceiro valor inteiro: resultado. Verificar se resultado contém um valor maior ou igual a 10; caso verdade mostrar o conteúdo de resultado e de (resultado + 15); senão mostrar resultado e (resultado – -7).

```
#include<stdio.h>
#include<stdlib.h>
#include<locale.h>

// Exemplo 2 - usando Estrutura de Decisão Composta

main()
{
        setlocale(LC_ALL,"Portuguese");

// Definição de variáveis

   int valor1, valor2, resultado, wk_resultado;

   system("cls");

   printf("\n<<  Exemplo 2 da Aula 04_2 - Estrutura de Decisão  >>");

// entrada de dados

   printf("\n\n Informe um primeiro Valor numérico inteiro: ");
   scanf("%d",&valor1);

   printf("\n\n Informe um segundo  Valor numérico inteiro: ");
   scanf("%d",&valor2);

// processamento do cálculo

   resultado = valor1 + valor2;
   wk_resultado = resultado;

// saída de dados

   if (resultado >= 10)
        {
            printf("\n\n O Resultado de valor1 + valor2 é: %d\n",valor1 + valor2);
            printf("\n O Resultado de %d + 15 é: %d\n",resultado, resultado + 15);
        }
   else

        {
            printf("\n\n O Resultado de valor1 + valor2 é: %d\n",valor1 + valor2);
            printf("\n O Resultado de %d - -7 é: %d\n",resultado, resultado - -7);
        }

   return 0;
}
```

3. Receber um valor inteiro: valor1, e verificar se o mesmo está na faixa de 20 até 100, inclusive. Caso o valor fornecido esteja nessa faixa de valores, então, multiplique esse valor por 10 e mostre o resultado, caso contrário mostre a mensagem: **"O número não está na faixa de 20 até 100, inclusive"**.

```
Algoritmo TestaValores;
Use Monitor de Vídeo;
Variáveis
    Valor1, Resultado : Numérico Inteiro;

Inicio
    Limpe a tela do monitor de vídeo;
    Escreva('Informe um valor inteiro: ');
    Leia(Valor1);
    Escreva;

    SE (Valor1 >= 20)  e (Valor1 <= 100) então
       Inicio
            Resultado ← Valor1  x  10;
            Escreva('O Resultado é: ',Resultado);
       Fim
    senão
        Escreva('O número não está na faixa de 20 a 100);
Fim.
```

2. Exercícios de fixação (Estrutura de Decisão Composta):

1. A SABESP – Companhia de Saneamento Básico de São Paulo, recebe em sua estação central de tratamento de água, volumes de água oriundos das represas Billings, Guarapiranga e Itupararanga. O volume normal de água recebido pela estação é de 100 milhões de litros por dia.

   Construir um algoritmo, em linguagem C++, para:

   a. Receber a informação sobre o volume de água recebido pela estação (definir essa variável de entrada como sendo do tipo de dado Inteiro).

   b. Baseando-se no volume informado, mostrar em tela a mensagem correspondente, conforme tabela abaixo.

Volume de água captado (milhões de litros)	Mensagem
100 ou superior	**A captação diária de água está normal.**
de 80 a 99	**Aumentar a vazão da Billings em 20%.**
de 60 a 79	**Aumentar a vazão da Guarapiranga em 15%.**
de 40 a 59	**Aumentar a vazão da Itupararanga em 30%.**
Abaixo de 40	**Iniciar programa de racionamento de água na grande São Paulo.**

2. Construir um programa, em linguagem C++, para:

   Sendo informado um determinado valor em Reais (R$) e o código da moeda, efetuar a conversão desse valor para a moeda informada, conforme tabela abaixo. Mostrar a Descrição da moeda e o valor convertido.

Código da moeda	Descrição da moeda	Fator de conversão em R$
1	**US Dollar**	**2,48**
2	**Peso Argentino**	**0,45**
3	**Euro**	**3,15**
4	**Ien**	**2,20**
Outro valor qualquer	**Código indeterminado**	

3. Construir um programa, em linguagem C++, para:

   Dado um número inteiro de segundos determine o seu valor equivalente em Graus, Minutos e Segundos. Se a quantidade de segundos for insuficiente para dar o valor em Graus, o valor em Graus deve ser Zero. A mesma observação continua válida em relação a minutos e segundos.

4.5 Estrutura de Decisão Encadeada

Uma **Estrutura de decisão encadeada ocorre** quando um determinado **comando** (ação)**, ou bloco de comandos** (ações)**,** deve ser executado **se um grande conjunto de possibilidades** ou **combinações de situações (expressões lógicas) for satisfeito (verdade)**.

A estrutura de decisão encadeada é expressa em agrupamento de várias estruturas de decisão, do tipo – **SE... Então** e/ou **SE... Então... Senão...**.

Atenção! O uso dos operadores lógicos **(E, Ou e Não),** em muitos casos **evita a utilização** de muitas estruturas **do tipo: SE... Então...**, encadeadas.

Exemplo de Estrutura de Decisão Encadeada Homogênea

Uma **Estrutura de Decisão Encadeada** é considerada **homogênea** quando segue um **padrão lógico de construção**, tornando pouco complexo sua leitura e compreensão.

Modelo de uma Estrutura de Decisão Encadeada Homogênea:

```
SE  <condição1>  Então
    SE  <condição 2>  Então
        SE  < condição 3 >  Então
            Inicio                          // Inicio do Bloco  Verdade
                |     <instrução simples 1>;
                |     <instrução simples 2>;
                |     <instrução simples 3>;
             Fim;                           // Fim do bloco verdade
```

Essa estrutura poderia ser **melhorada logicamente (refinada)** usando o operador lógico **E,** conforme mostrado a seguir:

```
SE  <condição1>  E  <condição 2>  E  < condição 3 >  então
    Inicio                          // Inicio do Bloco  Verdade
        <instrução simples 1>;
        <instrução simples 2>;
        <instrução simples 3>;
    Fim;                            // Fim do bloco verdade
```

Essa estrutura em linguagem C++:

```
if  <condição1>  &&  <condição 2>  &&  < condição 3 >
    {                               // Inicio do Bloco  Verdade
        <instrução simples 1>;
        <instrução simples 2>;
        <instrução simples 3>;
    }                               // Fim do bloco verdade
```

Um exemplo de programa usando uma Estrutura de Decisão Encadeada Homogênea foi dado anteriormente, para determinar o Tempo de Caso de uma pessoa, e qual a Bodas que está comemorando.

☞ **Estrutura de Decisão Encadeada Heterogênea**

É quando uma **Estrutura de Decisão Encadeada não segue um padrão lógico de construção**, devido a um grande conjunto de possibilidades ou combinações de situações a serem satisfeitas.

Essa estrutura, em muitos casos, é **de difícil leitura** e sua compreensão é demorada.

Modelo de uma Estrutura de Decisão Encadeada Heterogênea:

```
SE  <condição1>  Então
  Inicio
    SE  <condição 2>  Então
        Inicio                                  // Inicio do Bloco  verdade 1
           |      <instrução simples 1>;
           |      <instrução simples 2>;
           |      <instrução simples 3>;
        Fim;                                    // Fim do bloco verdade 1
  Fim
Senão
     SE  < condição 3 >  Então
        Inicio                                  // Inicio do Bloco  verdade 2
           |      <instrução simples 4>;
           |      <instrução simples 5>;
           |      <instrução simples 6>;
        Fim                                     // Fim do bloco verdade 2
     Senão
        SE  < condição 4 >  Então
           SE  < condição 5 >  Então
                    <instrução simples 7>       // comando Verdade
           Senão
                    <instrução simples 8>;      // Comando Falso
```

Esse tipo de estrutura poderá ser exigida em determinados exercícios, nesta disciplina.

📖 **Bons Estudos!**